

Bài 3: TÍNH GẦN ĐÚNG NGHIỆM PHƯƠNG TRÌNH

Nhóm:

TT	Họ và tên	MSSV	Lớp	Ghi chú
1	Lê Nguyễn Thùy Dương	232307023	23DTV-CLC3	
2	Phạm Minh Ngọc	23207012	23DTV-CLC3	

Bài 1

Code bài làm

```
syms x;
fx = x^2 - 2*sin(x) + 1/2;
fxi = matlabFunction(fx);
y = fxi(2);
fprintf('Ket qua cua phuong trinh: %.5f\n', y);
```

Hình chụp kết quả minh họa

```
>> Lab3_1
Ket qua cua phuong trinh: 2.68141
```

Bài 2

Code phương pháp chia đôi

```
function [c,n] = chiadoi(fx, a, b, saiso)
fxi = str2func(['@(x)', fx]);
n = 0;
while(1)
    c = (a+b)/2;
    if(fxi(a)*fxi(c) < 0)
        b = c;
    else
        a = c;
    end
    n = n + 1;
    if abs(b - a) < saiso
        break;
    end
end
end
```

Code phương pháp lặp

```
function [x1,n] = lap(fx, fp, a, b, saiso)
n = 0;
x0 = b;
fpi = str2func(['@(x)', fp]);
while(1)
    x1 = fpi(x0);
    n = n + 1;
    if abs(x1 - x0) < saiso
```

```

        break;
    end
    x0 = x1;
end
end

```

Code phương pháp tiếp tuyến

```

function [x1,n]=tieptuyen(fx, a, b, saiso)
syms x;
f_sym = str2sym(fx);
df_sym = diff(f_sym, x);

f = matlabFunction(f_sym);
df = matlabFunction(df_sym);

x0 = b;
n = 0;
while(1)
    x1 = x0 - (f(x0)/df(x0));
    n = n + 1;
    if abs(x1 - x0) < saiso
        break;
    end
    x0 = x1;
end
end

```

Code phương pháp dây cung

```

function [x1,n]=daycung(fx, a, b, saiso)
f = str2func(['@(x)', fx]);
n = 0;
while(1)
    c = (a*f(b) - b*f(a))/(f(b) - f(a));
    a = b;
    b = c;
    n = n + 1;
    if abs(b - a) < saiso
        break;
    end
end
x1 = c;
end

```

a. Code bài làm

```

a = -1;
b = 2;
ss = 0.005;
fx = 'x - sin(x) - 0.25';
[nghiem,solanlap] = chiadoi(fx, a, b, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);

```

Hình chụp kết quả minh họa

```
>> Lab3_2a_chiadoi
Nghiem gần đúng của phương trình: 1.17090
Số lần lặp: 10
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
fp = 'sin(x)+0.25';
a = -1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = lap(fx, fp, a, b, ss);
fprintf('Root found: %.5f\n', nghiem);
fprintf('Number of iterations: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_lap
Nghiem gần đúng của phương trình: 1.16939
Số lần lặp: 3
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
a = -1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = tieptuyen(fx, a, b, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_tieptuyen
Nghiem gần đúng của phương trình: 1.17123
Số lần lặp: 4
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
a = -1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = daycung(fx, a, b, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_daycung
Nghiem gần đúng của phương trình: 1.17134
Số lần lặp: 9
```

Từ 4 phương pháp ta thấy được **phương pháp chia đôi có số lần lặp nhiều nhất** (10 lần). Ở phương pháp chia đôi chỉ sử dụng giá trị đầu của hàm tại hai đầu đoạn nên tốc độ hội tụ tuyến tính (chậm). Còn ở phương pháp lặp thì có thể nhanh hơn những phương pháp khác nếu chọn được hàm $g(x)$ phù hợp nhưng thỉnh thoảng vẫn xảy ra trường hợp hội tụ sai do chọn sai hàm. Với phương pháp tiếp tuyến thì hội tụ rất nhanh do dùng đạo hàm để dự đoán vị trí nghiệm. Cuối cùng là phương pháp dây cung, phương pháp này đảm bảo độ ổn định nên tốc độ hội tụ cũng nằm ở mức trung bình.

b. Code bài làm

```
a = 1;
b = 2;
ss = 0.005;
fx = 'x - sin(x) - 0.25';
[nghiem,solanlap] = chiadoi(fx, a, b, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_chiadoi
Nghiệm gần đúng của phương trình: 1.16797
Số lần lặp: 8
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
fp = 'sin(x)+0.25';
a = 1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = lap(fx, fp, a, b, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_lap
Nghiệm gần đúng của phương trình: 1.16939
Số lần lặp: 3
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
a = 1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = tieptuyen(fx, a, b, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_tieptuyen
Nghiem gần đúng của phương trình: 1.17123
Số lần lặp: 4
```

Code bài làm

```
fx = 'x - sin(x) - 0.25';
a = 1;
b = 2;
ss = 0.005;
[nghiem, solanlap] = daycung(fx, a, b, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem);
fprintf('Số lần lặp: %d\n', solanlap);
```

Hình chụp kết quả minh họa

```
>> Lab3_2a_daycung
Nghiem gần đúng của phương trình: 1.17119
Số lần lặp: 4
```

Phương pháp lặp có số lần lặp ít nhất. Vì ở đoạn $[1;2]$ $g(x) = \sin(x) + 0,25 \Rightarrow |g'(x)| = |\cos(x)| < 1$ và gần nghiệm nên hội tụ rất tốt. Còn phương pháp tiếp tuyến hiện không nhanh bằng phương pháp lặp là do chọn điểm nghiệm đầu tiên xa nghiệm gần đúng nên cần nhiều vòng lặp hơn. Nếu hiệu chỉnh lại hàm của phương pháp tiếp tuyến lấy nghiệm đầu tiên là $a = 1$ thì số lần lặp của phương pháp tiếp tuyến bằng với phương pháp lặp.

c.

Code bài làm

```
a = -0.5;
b = 0.5;
c = -0.05;
d = 0.5;
ss = 5e-5;
fx = 'x^2 - sin(x)';
[nghiem1, solanlap1] = chiadoi(fx, a, b, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem1);
fprintf('Số lần lặp (khoảng phân ly 1): %d\n', solanlap1);
[nghiem2, solanlap2] = chiadoi(fx, c, d, ss);
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem2);
fprintf('Số lần lặp (khoảng phân ly 2): %d\n', solanlap2);
```

Hình chụp kết quả minh họa

```
>> Lab3_2c_chiadoi
Nghiệm gần đúng của phương trình: 0.49997
Số lần lặp (khoảng phân ly 1): 15
Nghiệm gần đúng của phương trình: -0.00002
Số lần lặp (khoảng phân ly 2): 14
```

Trường hợp phương pháp chia đôi ra 2 nghiệm khác nhau. Ở khoảng phân ly $[-0.5; 0.5]$ thì $c = (a+b)/2 \Rightarrow c = 0$ và $f(a)*f(c) = 0$ nên $a = c$. Vậy nên $a = 0$ và $b = 0.5$. Lần lượt chạy tiếp những lần sau để $\text{abs}(b-a) < \text{saiso}$ nên khoảng nghiệm ngày càng bị dịch sang phía bên phải do vậy kết quả nghiệm bị sai.

Code bài làm

```
a = -0.5;
b = 0.5;
c = -0.05;
d = 0.5;
ss = 5e-5;
fx = 'x^2 - sin(x)';
[nghiem1, solanlap1] = daycung(fx, a, b, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem1);
fprintf('Số lần lặp (khoảng phân ly 1): %d\n', solanlap1);
[nghiem2, solanlap2] = daycung(fx, c, d, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem2);
fprintf('Số lần lặp (khoảng phân ly 2): %d\n', solanlap2);
```

Hình chụp kết quả minh họa

```
>> Lab3_2c_daycung
Nghiệm gần đúng của phương trình: -0.00000
Số lần lặp (khoảng phân ly 1): 8
Nghiệm gần đúng của phương trình: 0.00000
Số lần lặp (khoảng phân ly 2): 6
```

Ở khoảng phân ly bé hơn thì số lần lặp ít hơn

Code bài làm

```
a = -0.5;
b = 0.5;
c = -0.05;
d = 0.5;
ss = 5e-5;
fx = 'x^2 - sin(x)';
[nghiem1, solanlap1] = tieptuyen(fx, a, b, ss);
fprintf('Nghiệm gần đúng của phương trình: %.5f\n', nghiem1);
fprintf('Số lần lặp (khoảng phân ly 1): %d\n', solanlap1);
[nghiem2, solanlap2] = tieptuyen(fx, c, d, ss);
```

```
fprintf('Nghiem gần đúng của phương trình: %.5f\n', nghiem2);  
fprintf('Số lần lặp (khoảng phân ly 2): %d\n', solanlap2);
```

Hình chụp kết quả minh họa

```
>> Lab3_2c_tieptuyen  
Nghiem gần đúng của phương trình: 0.00000  
Số lần lặp (khoảng phân ly 1): 1  
Nghiem gần đúng của phương trình: -0.00000  
Số lần lặp (khoảng phân ly 2): 5
```

Do ở khoảng $[-0.5; 0.5]$ thì $x_0 = (a+b)/2 = 0$ (bằng đúng nghiệm của phương trình) nên chỉ cần chạy 1 lần lặp duy nhất. Còn ở khoảng phân ly còn lại thì phải chạy thêm nhiều lần hơn để có nghiệm gần đúng chính xác.